
SHARED MEMORY

Recall : What is a semaphore?

- Synchronization tool for processes to synchronize their activities
- Semaphore S – integer variable
- Can only be accessed via two indivisible (atomic) operations

```
wait(S)
{    while (S <= 0)
       ; // busy wait
     S--;
}
```

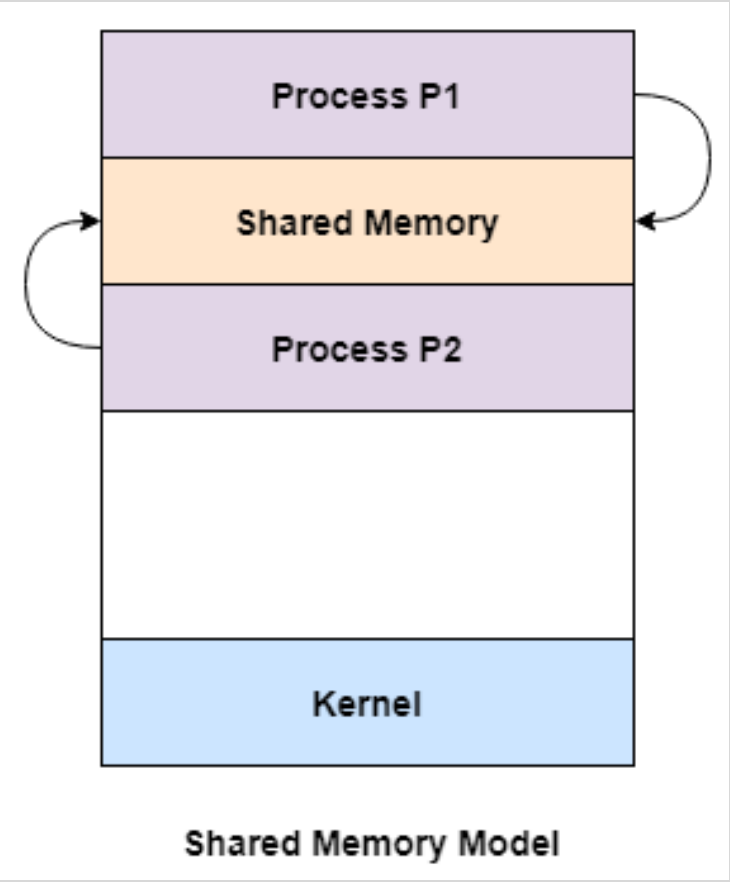
```
signal(S)
{
          S++;
}
```

Binary vs Counting Semaphores

- Binary Semaphore:
  - 0 or 1
  - Mutual Exclusion
  - One process can access a resource at a time
  - Example: Protecting a shared variable from being accessed by multiple processes simultaneously
- Counting Semaphore:
  - Non-negative integer-> Number of available resources
  - It can allow more than one process to access a resource concurrently

Shared memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.



Shared Memory Model

Adding System Calls for Counting Semaphores

- int sem_init(int sem_id, int value):
  - Initializes a semaphore with a specific ID and count.
- int sem_down(int sem_id):
  - Implements the P (wait) operation, decrementing the count and blocking the process if needed.
- int sem_up(int sem_id):
  - Implements the V (signal) operation, incrementing the count and waking a waiting process.

For Lab 3 you can also solve with a binary semaphore

Add a System Call for Shared Memory

- Add a global table of shared memory pages in the kernel.
  - Each entry stores: ID, physical page pointer.

- **Add two syscalls:**

  int shm_create(char *id)

  → allocates a shared page if not already created.

  char* shm_get(char *id)

  → maps existing page into caller's address space and returns pointer.

- Inside these syscalls:

  Use **kalloc()** to allocate a physical page.

  Use **mappages()** to map it into user space.

Global Table

- Shared Memory gives both processes access to the same physical page of memory. It's like giving them both a marker and letting them write on the same whiteboard.
  - The kernel creates a **global table** to keep track of all shared memory pages. Each entry in this table has:

    -> id: Name

    -> Page(Physical) : The actual physical memory location

    -> How may processes are accessing/ looking it (count)

int shm_create(char *id)

→ allocates a shared page if not already created.

- What does this mean?
- It checks the global table. If the ID is new, it allocates a physical page. If it exists, it uses the existing one.
- It then maps this physical page into the process's virtual address space so the process can use it.

char* shm_get(char *id)

→ maps existing page into caller's address space and returns pointer.

- When a different process wants to access the shared page, this function simply looks up the ID in the table and maps the **existing physical page** into its own virtual address space.

kalloc()

- Xv6/kalloc.c
- Physical memory allocator, intended to allocate memory for user processes.
- Allocates in 4096-byte "pages".
- Free list is kept sorted and combines adjacent pages into long runs, to make it easier to allocate big segments.
- Summary: This is a kernel function that allocates a single page (4096 bytes) of physical memory. It returns a pointer to the start of this page.

Github kalloc()

- Discussion on kalloc() from Github

mappages()

```
// Create PTEs for virtual addresses starting at va that refer to
// physical addresses starting at pa. va and size might not
// be page-aligned.
static int
mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm)
{
  char *a, *last;
  pte_t *pte;

  a = (char*)PGROUNDDOWN((uint)va);
  last = (char*)PGROUNDDOWN(((uint)va) + size - 1);
  for(;;){
    if((pte = walkpgdir(pgdir, a, 1)) == 0)
      return -1;
    if(*pte & PTE_P)
      panic("remap");
    *pte = pa | perm | PTE_P;
    if(a == last)
      break;
    a += PGSIZE;
    pa += PGSIZE;
  }
  return 0;
}
```

1. Align start VA to Page Boundary
2. Align end VA

3. Get or create PTE for VA
4. Already mapped, error
5. Map PA to VA with PERMS
6. Done
7. Next VA page
8. NEXT PA PAGE

Example: Map 8000 Bytes from VA 0x1234 to PA 0x5000

- VA= 0x1234
- Size = 8000 bytes
- PA = 0x5000

---

byte addressable

First byte is at address 0x1234

2nd Byte at address 0x1235

…

8000th Byte at address (0x1234+0x1F40) = 0x3174

(Simply convert 8000 to hexadecimal and add 1234)

Virtual memory region is from 0x1234 to 0x3174

1 page size is 4 KB

i.e 1st VA page is in 0x1000 to be mapped to 0x5000

2nd VA page is in 0x2000 to be mapoped to 0x6000

3rd VA page is in 0x3000 to be mapped to 0x7000

Thank You